

AXI4-Lite

SPI / I2C IP 설계

& UVM 기능 검증

AXI4-Lite

SPI

I2C

MicroBlaze

Vitis

UVM

VCS/Verdi

FPGA 임베디드 시스템 설계 프로젝트

MicroBlaze → AXI Slave → SPI/I2C Master → Slave FPGA

발표자 : 김민기

01

AXI4-Lite 프로토콜 개요

5-Channel 구조, 핸드셰이크, Write/Read 트랜잭션

02

IP 레지스터 맵

AXI4-Lite SPI Slave / I2C Slave / GPIO Slave 레지스터 구조

03

Vitis SW — Layered Architecture

Application / Driver / HAL / HW 4계층 설계

04

하드웨어 동작 시연

SPI & I2C FND 구동 영상

05

UVM 검증 환경

Test / Env / Agent / Scoreboard / Coverage 구조

06

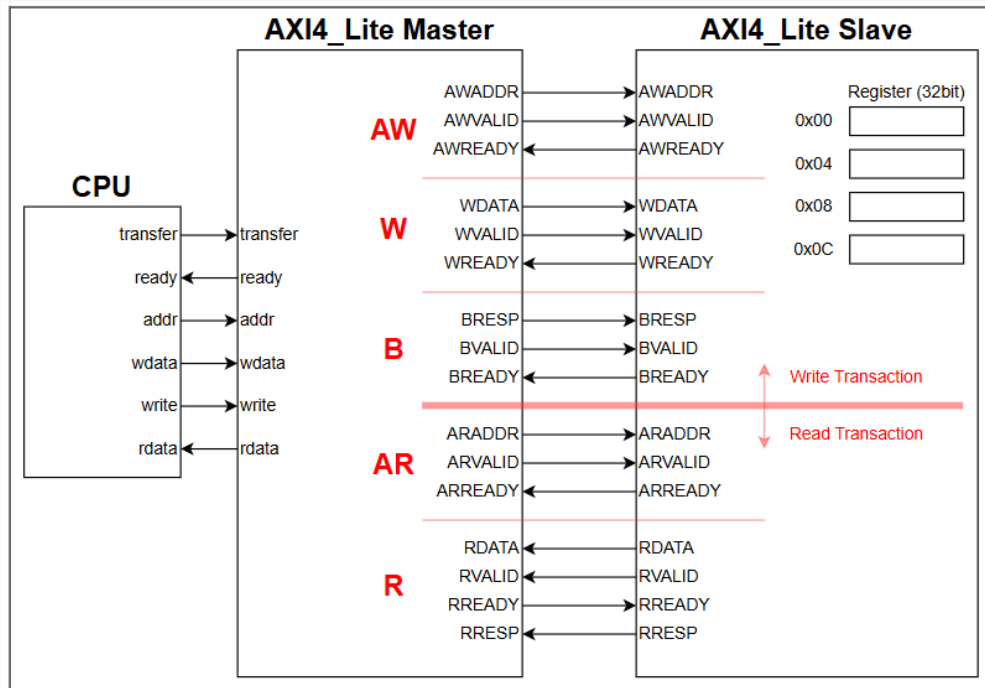
Verdi 파형 & VCS 결과

Pass Log, Functional Coverage

07

Trouble Shooting & 느낀 점

주요 이슈 해결 및 느낀 점



AW

쓰기 주소 채널
AWADDR · AWVALID · AWREADY

W

쓰기 데이터 채널
WDATA ·WSTRB · WVALID · WREADY

B

쓰기 응답 채널
BRESP · BVALID · BREADY → OKAY

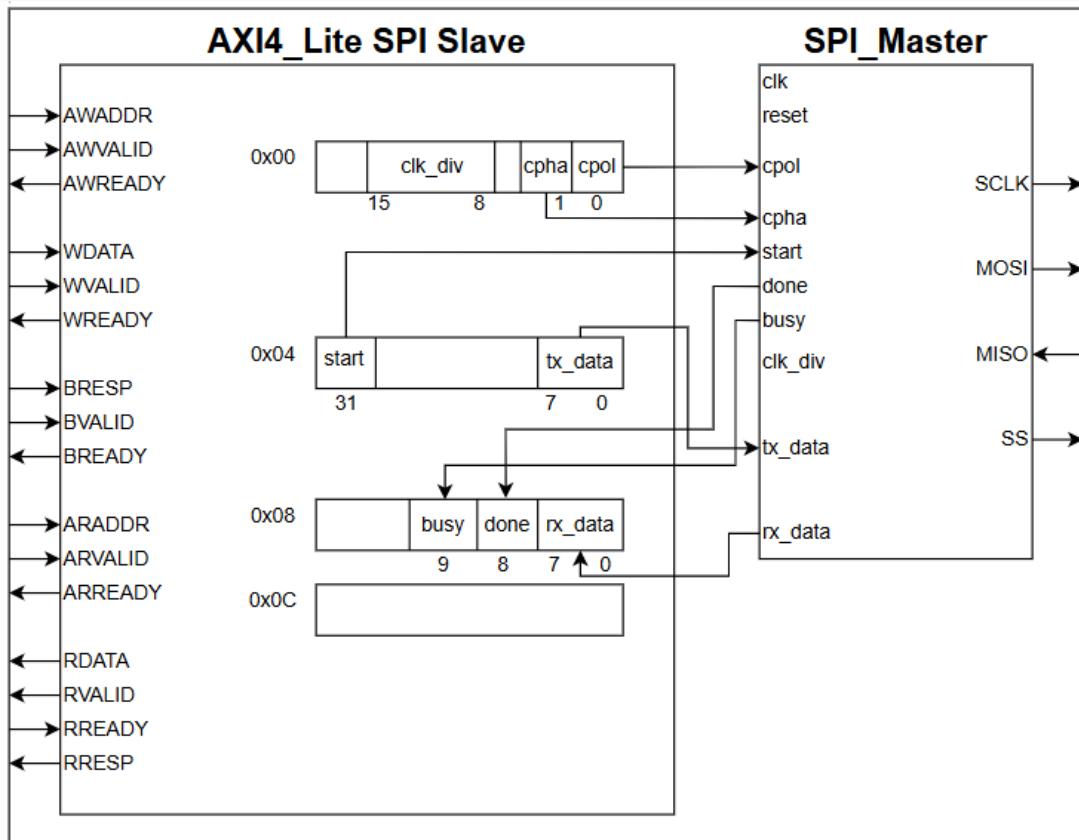
AR

읽기 주소 채널
ARADDR · ARVALID · ARREADY

R

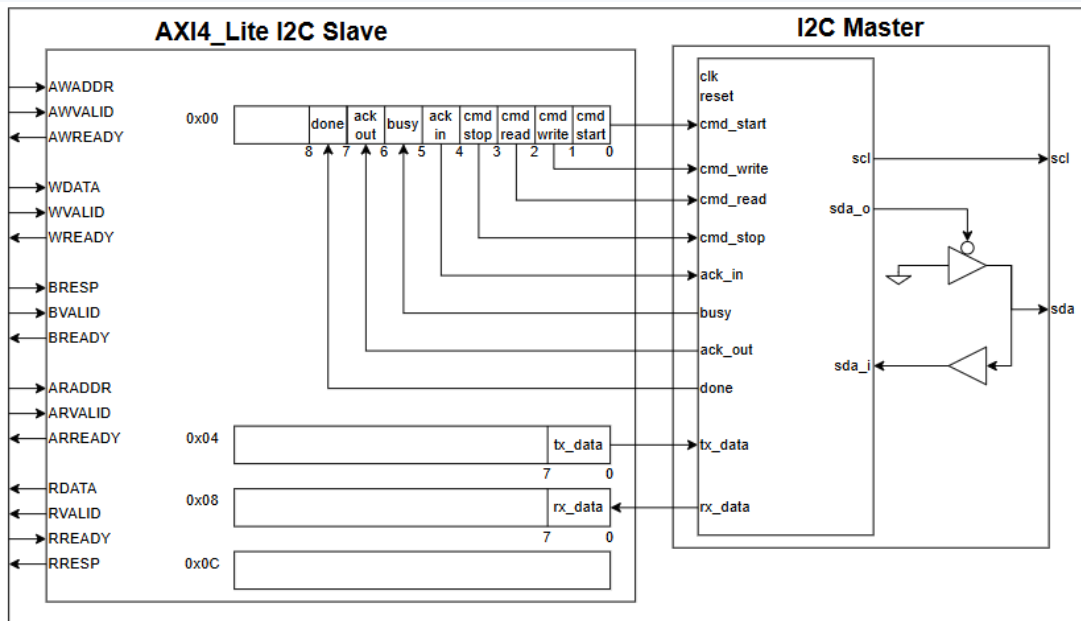
읽기 데이터 채널
RDATA · RRESP · RVALID · RREADY

핸드셰이크 성립 : VALID & READY 동시 High



레지스터 맵 요약

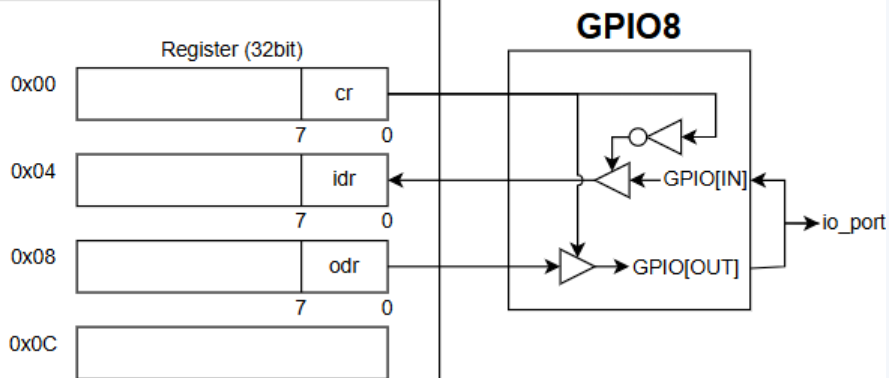
주소	필드	설명
0x00	clk_div[15:8]	SPI 클럭 분주비
0x00	cpha[1], cpol[0]	SPI Mode 설정
0x04	start[31]	전송 시작 트리거
0x04	tx_data[7:0]	송신 데이터
0x08	busy[9]	전송 중 플래그
0x08	done[8]	전송 완료
0x08	rx_data[7:0]	수신 데이터
0x0C	—	예약



레지스터 맵 요약

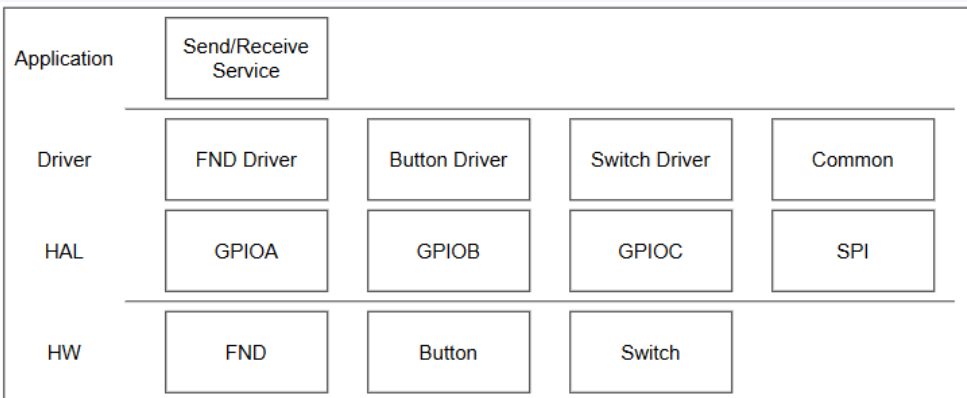
주소	필드	설명
0x00	cmd_start[0]	START 조건 발행
0x00	cmd_write[1]	쓰기 커맨드
0x00	cmd_read[2]	읽기 커맨드
0x00	cmd_stop[3]	STOP 조건 발행
0x00	busy[5]	버스 사용 중
0x00	ack_in[4]	ACK 수신 여부
0x00	ack_out[6]	ACK 송신
0x04	tx_data[7:0]	송신 데이터
0x08	rx_data[7:0]	수신 데이터

AXI4_Lite GPIO8 Slave



레지스터 맵 요약

주소	필드	설명
0x00	cr[7:0]	Control Register 방향 제어 (1=출력)
0x04	idr[7:0]	Input Data Register GPIO 입력값 읽기
0x08	odr[7:0]	Output Data Register GPIO 출력값 쓰기
0x0C	—	예약



Application Layer

Send/Receive Service
main() — 통신 시나리오, FND 패턴 전송 로직



Driver Layer

FND Driver · Button Driver
Switch Driver · Common — 프로토콜 추상화



HAL Layer

GPIOA/B/C · SPI



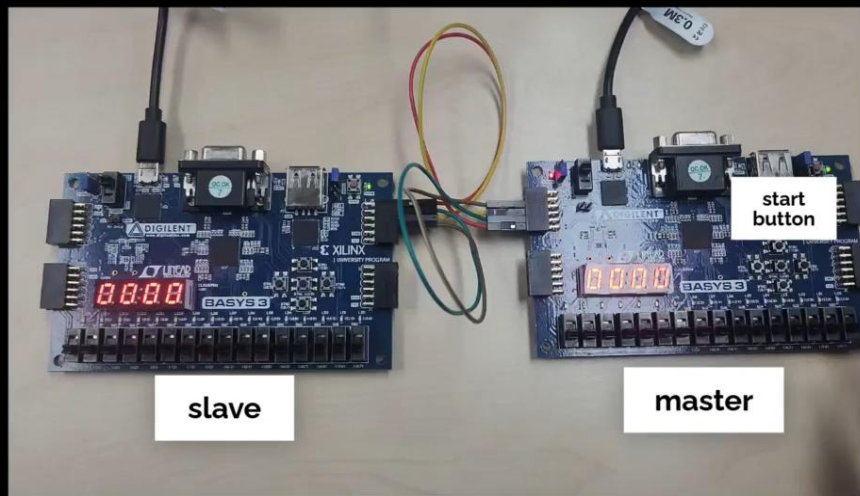
HW Layer

FND · Button · Switch
AXI4-Lite IP (Vivado 합성)

↑ 상위 계층은 하위 계층 함수만 호출 | HW 교체 시 HAL만 수정 → 상위 코드 재사용 가능

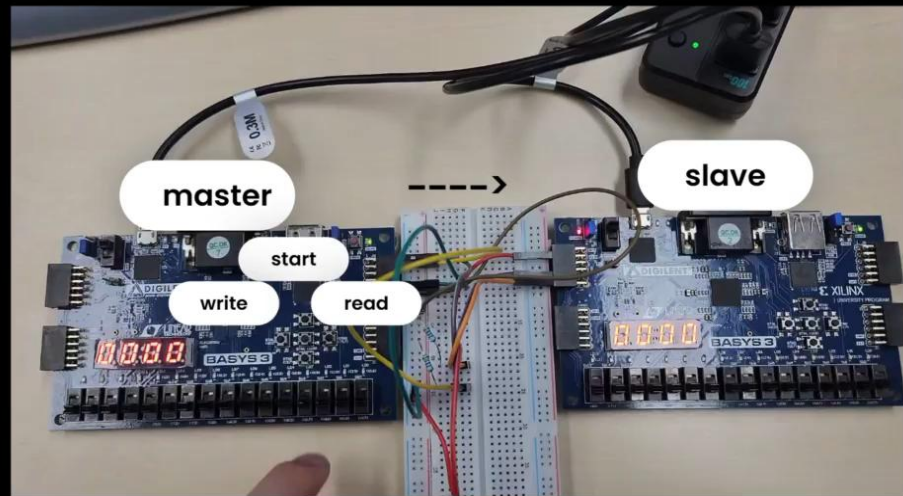
SPI 통신 시연

SPI
master → slave



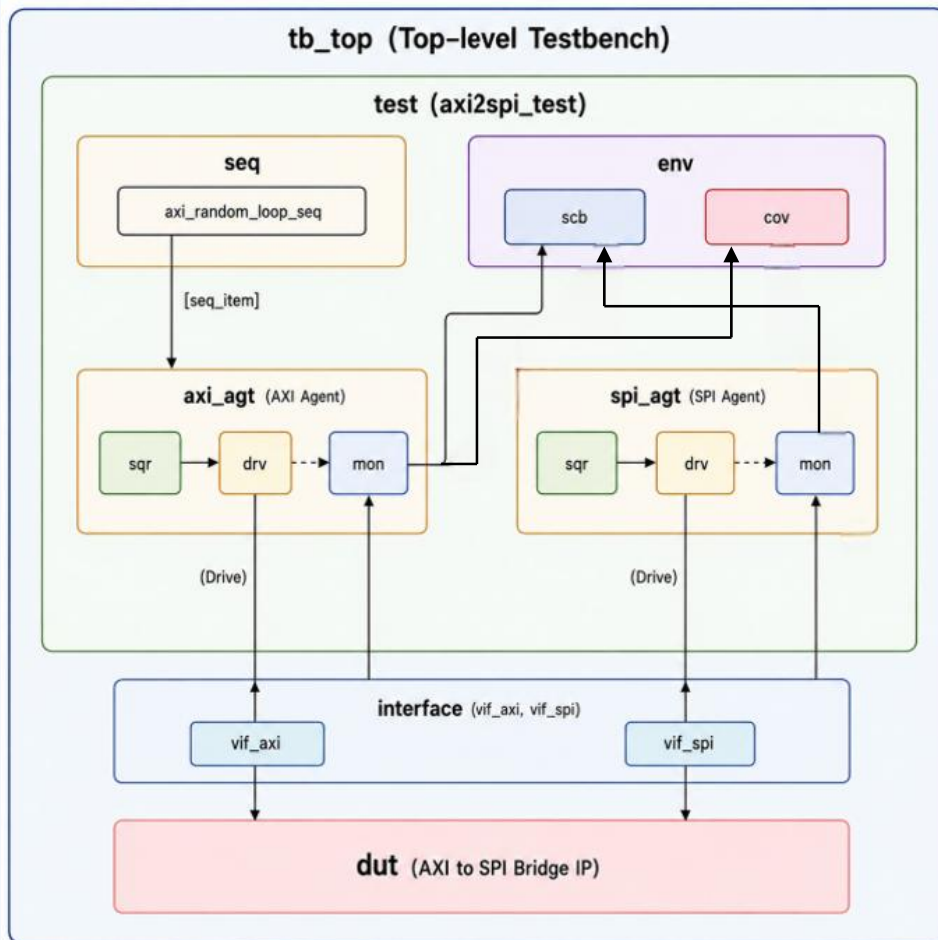
I2C 통신 시연

I2C
master → slave



```

_top
agents
├── axi_agent
│   ├── axi_agent.sv
│   ├── axi_agent_pkg.sv
│   ├── axi_driver.sv
│   ├── axi_monitor.sv
│   ├── axi_seq_item.sv
│   └── axi_sequencer.sv
├── spi_agent
│   ├── spi_agent.sv
│   ├── spi_agent_pkg.sv
│   ├── spi_driver.sv
│   ├── spi_monitor.sv
│   ├── spi_seq_item.sv
│   └── spi_sequencer.sv
└── axi2spi_test.sv
env
├── axi2spi_coverage.sv
├── axi2spi_env.sv
├── axi2spi_scoreboard.sv
└── env_pkg.sv
sequences
├── axi_random_loop_seq.sv
├── axi_read_seq.sv
├── axi_write_read_seq.sv
├── axi_write_seq.sv
├── spi_dummy_seq.sv
├── spi_if.sv
├── tb_axi4_spi.sv
└── test_pkg.sv
  
```



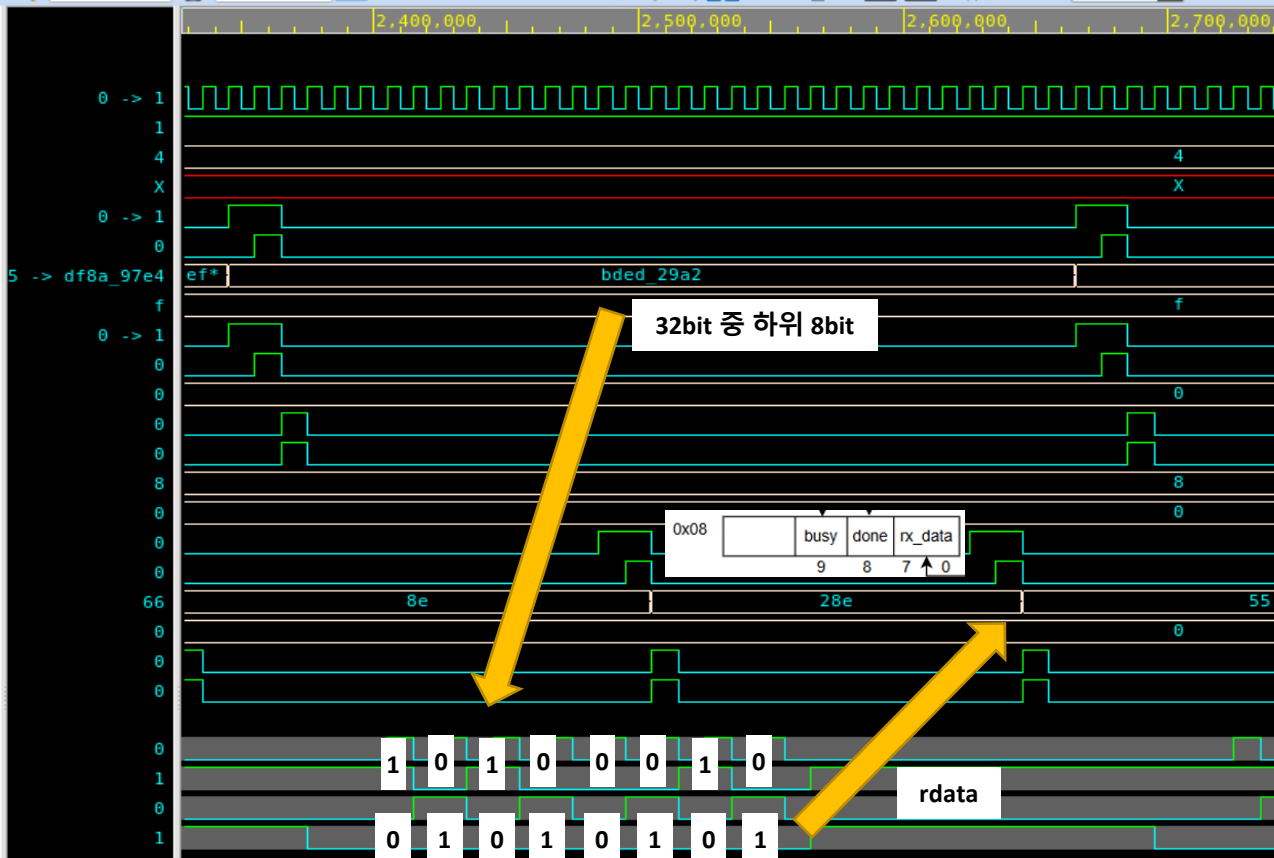
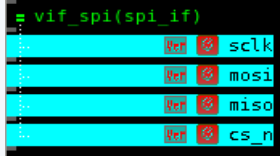
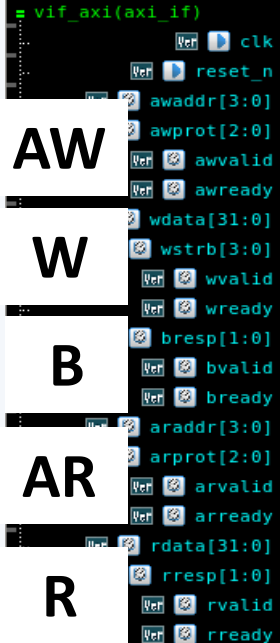
axi_agt

AXI Master 역할

spi_agt

SPI Slave 역할

R



VCS Simulation Log

```

UVM_INFO ./tb/sequences/axi_random_loop_seq.sv(13) @ 319775000: uvm_test_top.env.m_axi_agent.sequencer@@axi_seq [SEQ] --- Iteration 1000/1000 ---
UVM_INFO ./tb/agents/axi_agent/axi_monitor.sv(36) @ 319805000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Write: ADDR=4 DATA=a0edb081
UVM_INFO ./tb/agents/axi_agent/axi_driver.sv(55) @ 319915000: uvm_test_top.env.m_axi_agent.driver [DRV] Driving READ: ADDR=0x8
UVM_INFO ./tb/agents/axi_agent/axi_monitor.sv(55) @ 319955000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Read: ADDR=8 RDATA=256
UVM_INFO ./tb/agents/spi_agent/spi_monitor.sv(42) @ 320005000: uvm_test_top.env.m_spi_agent.monitor [SPI MON] Captured MOSI = 0x81, MISO = 0x17
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(46) @ 320005000: uvm_test_top.env.m_scoreboard [SCB] [PASS] WRITE: AXI(0x81) == SPI_MOSI(0x81)
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(68) @ 320005000: uvm_test_top.env.m_scoreboard [SCB] Waiting for SPI to finish (busy=1)...
UVM_INFO ./tb/agents/axi_agent/axi_driver.sv(55) @ 320055000: uvm_test_top.env.m_axi_agent.driver [DRV] Driving READ: ADDR=0x8
UVM_INFO ./tb/agents/axi_agent/axi_monitor.sv(55) @ 320095000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Read: ADDR=8 RDATA=17
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(73) @ 320095000: uvm_test_top.env.m_scoreboard [SCB] [PASS] READ : AXI RX(0x17) == Expected MISO(0x17)

```

Functional Coverage

```

// 1. 송신 데이터 (AXI -> SPI) 커버포인트
cp_tx_data: coverpoint cov_tx_data {
  bins zero      = {8'h00};
  bins max       = {8'hFF};
  bins alt_55    = {8'h55};
  bins alt_AA    = {8'hAA};
  bins low_rng   = {[8'h01 : 8'h7F]};
  bins high_rng  = {[8'h80 : 8'hFE]};
}

// 2. 수신 데이터 (SPI -> AXI) 커버포인트
cp_rx_data: coverpoint cov_rx_data {
  bins zero      = {8'h00};
  bins max       = {8'hFF};
  bins alt_55    = {8'h55};
  bins alt_AA    = {8'hAA};
  bins low_rng   = {[8'h01 : 8'h7F]};
  bins high_rng  = {[8'h80 : 8'hFE]};
}

cross_tx_rx: cross cp_tx_data, cp_rx_data;

```

```

uvm_test_top.env.m_coverage [COV]

uvm_test_top.env.m_coverage [COV] ===== AXI2SPI Coverage Summary =====
uvm_test_top.env.m_coverage [COV] Overall Coverage      : 100.0%
uvm_test_top.env.m_coverage [COV] TX Data Coverage   : 100.0%
uvm_test_top.env.m_coverage [COV] RX Data Coverage   : 100.0%
uvm_test_top.env.m_coverage [COV] Cross (TX x RX)    : 100.0%
uvm_test_top.env.m_coverage [COV] =====

0: uvm_test_top.env.m_scoreboard [SCB]

0: uvm_test_top.env.m_scoreboard [SCB] ===== AXI2SPI Scoreboard Summary =====
0: uvm_test_top.env.m_scoreboard [SCB] Total checks : 2000
0: uvm_test_top.env.m_scoreboard [SCB] Pass         : 2000
0: uvm_test_top.env.m_scoreboard [SCB] Fail         : 0
0: uvm_test_top.env.m_scoreboard [SCB] =====

```

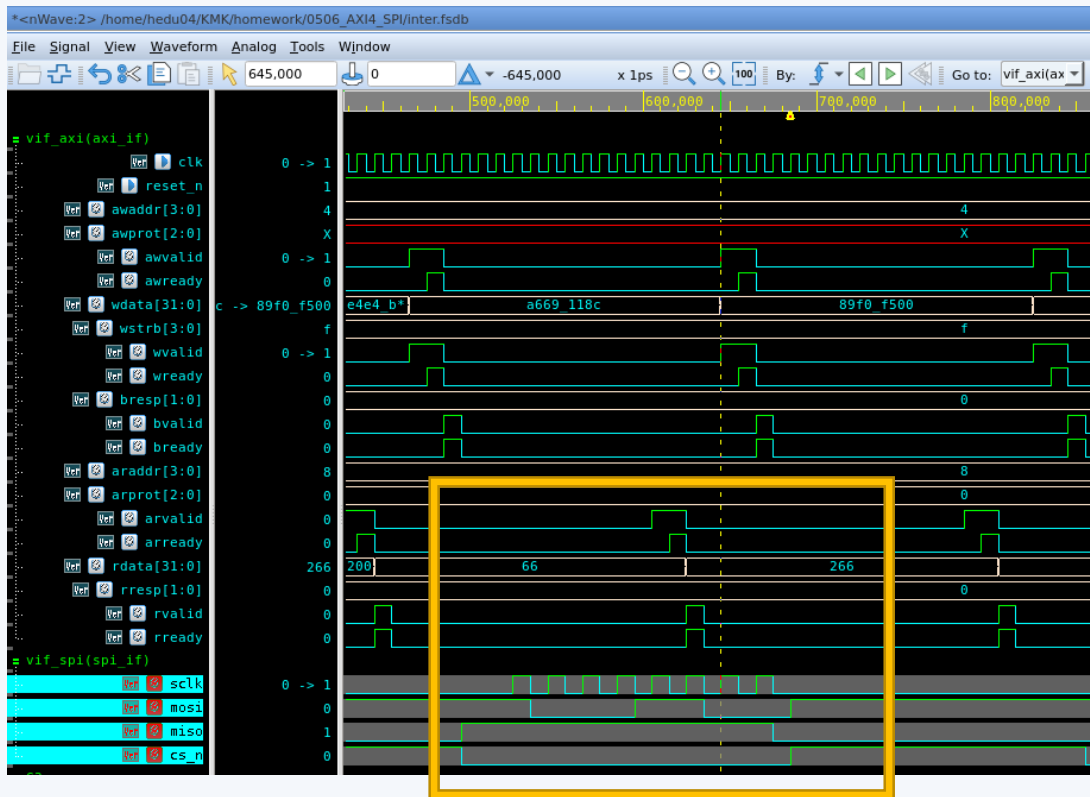
```

UVM_INFO ./tb/sequences/axi_random_loop_seq.sv(13) @ 179915000: uvm_test_top.env.m_axi_agent.sequencer@@axi_seq [SEQ] --- Iteration 1000/1000 ---
UVM_INFO ./tb/agents/axi_agent/axi_monitor.sv(36) @ 179945000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Write: ADDR=4 DATA=8edbd555
UVM_INFO ./tb/agents/spi_agent/spi_monitor.sv(42) @ 179965000: uvm_test_top.env.m_spi_agent.monitor [SPI_MON] Captured MOSI = 0x0, MISO = 0x86
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(46) @ 179965000: uvm_test_top.env.m_scoreboard [SCB] [PASS] WRITE: AXI(0x0) == SPI_MOSI(0x0)
UVM_ERROR ./tb/env/axi2spi_scoreboard.sv(78) @ 179965000: uvm_test_top.env.m_scoreboard [SCB] [FAIL] READ : AXI_RX(0xf4) != Expected_MISO(0x86)
UVM_INFO ./tb/agents/axi_agent/axi_driver.sv(55) @ 180055000: uvm_test_top.env.m_axi_agent.driver [DRV] Driving READ: ADDR=0x8
UVM_INFO ./tb/agents/axi_agent/axi_monitor.sv(55) @ 180095000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Read: ADDR=8 RDATA=86
UVM_INFO /tools/synopsys/vcs/W-2024.09-SP1/etc/uvm-1.2/base/uvm_objection.svh(1276) @ 181095000: reporter [TEST_DONE] 'run' phase is ready to pro
to the 'extract' phase
UVM_INFO ./tb/env/axi2spi_coverage.sv(57) @ 181095000: uvm_test_top.env.m_coverage [COV]
=====
UVM_INFO ./tb/env/axi2spi_coverage.sv(58) @ 181095000: uvm_test_top.env.m_coverage [COV] ===== AXI2SPI Coverage Summary =====
UVM_INFO ./tb/env/axi2spi_coverage.sv(59) @ 181095000: uvm_test_top.env.m_coverage [COV] Overall Coverage : 100.0%
UVM_INFO ./tb/env/axi2spi_coverage.sv(61) @ 181095000: uvm_test_top.env.m_coverage [COV] TX Data Coverage : 100.0%
UVM_INFO ./tb/env/axi2spi_coverage.sv(63) @ 181095000: uvm_test_top.env.m_coverage [COV] RX Data Coverage : 100.0%
UVM_INFO ./tb/env/axi2spi_coverage.sv(65) @ 181095000: uvm_test_top.env.m_coverage [COV] Cross (TX x RX) : 100.0%
UVM_INFO ./tb/env/axi2spi_coverage.sv(67) @ 181095000: uvm_test_top.env.m_coverage [COV] =====
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(90) @ 181095000: uvm_test_top.env.m_scoreboard [SCB]
=====
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(91) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] ===== AXI2SPI Scoreboard Summary =====
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(92) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] Total checks : 750
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(93) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] Pass : 25
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(94) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] Fail : 725
UVM_INFO ./tb/env/axi2spi_scoreboard.sv(95) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] =====
UVM_ERROR ./tb/env/axi2spi_scoreboard.sv(97) @ 181095000: uvm_test_top.env.m_scoreboard [SCB] TEST FAILED!

```

문제 :

AXI 버스를 통해 Write 후 Read를 수행하는 시퀀스 동작 중, 스코어보드에서 FAIL이 발생함.



axi_random_loop_seq.sv

```

req = axi_seq_item::type_id::create("req");
start_item(req);
if (!req.randomize() with {
    dir == axi_seq_item::WRITE;
    addr == 4'h4;
    data[31] == 1'b1;})
    `uvm_fatal("SEQ", "Write Rand Fail")
finish_item(req);

#100;

req = axi_seq_item::type_id::create("req");
start_item(req);
if (!req.randomize() with { dir == axi_seq_item::READ; addr == 4'h8; })
    `uvm_fatal("SEQ", "Read Rand Fail")
finish_item(req);

```

원인 :

write, read 동작 사이 dut delay 대기

너무 짧은 delay로 인해

SPI 통신 중에 read를 함

axi_random_loop_seq.sv

```

req = axi_seq_item::type_id::create("req");
start_item(req);
if (!req.randomize() with {
    dir == axi_seq_item::WRITE;
    addr == 4'h4;
    data[31] == 1'b1;})
    `uvm_fatal("SEQ", "Write Rand Fail")
finish_item(req);

do begin
    #100;
    req = axi_seq_item::type_id::create("req");
    start_item(req);
    if (!req.randomize() with { dir == axi_seq_item::READ; addr == 4'h8; })
        `uvm_fatal("SEQ", "Read Rand Fail")
    finish_item(req);
end while (req.rdata[9] == 1'b1);

```

```

19775000: uvm_test_top.env.m_axi_agent.sequencer@axi_seq [SEQ] --- Iteration 1000/1000 ---
9805000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Write: ADDR=4 DATA=a0edb081
915000: uvm_test_top.env.m_axi_agent.driver [DRV] Driving READ: ADDR=0x8
9955000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Read: ADDR=8 RDATA=256
9005000: uvm_test_top.env.m_spi_agent.monitor [SPI_MON] Captured MOSI = 0x81, MISO = 0x17
0: uvm_test_top.env.m_scoreboard [SCB] [PASS] WRITE: AXI(0x81) == SPI_MOSI(0x81)
0: uvm_test_top.env.m_scoreboard [SCB] Waiting for SPI to finish (busy=1)...
955000: uvm_test_top.env.m_axi_agent.driver [DRV] Driving READ: ADDR=0x8
9095000: uvm_test_top.env.m_axi_agent.monitor [MON] Captured Read: ADDR=8 RDATA=17
0: uvm_test_top.env.m_scoreboard [SCB] [PASS] READ : AXI_RX(0x17) == Expected_MISO(0x17)
/base/uvm_objection.svh(1276) @ 321095000: reporter [TEST_DONE] 'run' phase is ready to pro

uvm_test_top.env.m_coverage [COV]

uvm_test_top.env.m_coverage [COV] ===== AXI2SPI Coverage Summary =====
uvm_test_top.env.m_coverage [COV] Overall Coverage : 100.0%
uvm_test_top.env.m_coverage [COV] TX Data Coverage : 100.0%
uvm_test_top.env.m_coverage [COV] RX Data Coverage : 100.0%
uvm_test_top.env.m_coverage [COV] Cross (TX x RX) : 100.0%
uvm_test_top.env.m_coverage [COV] =====

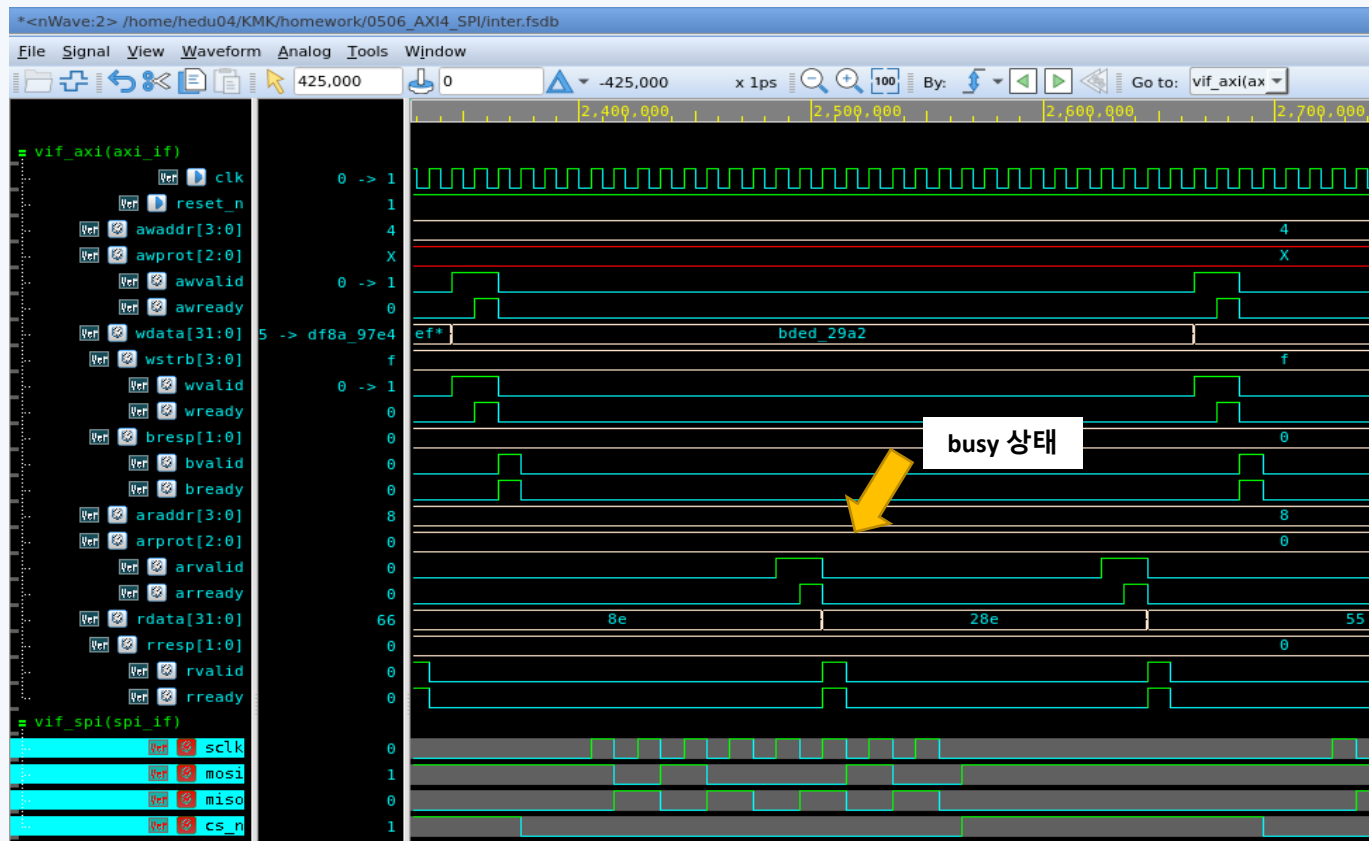
0: uvm_test_top.env.m_scoreboard [SCB]

0: uvm_test_top.env.m_scoreboard [SCB] ===== AXI2SPI Scoreboard Summary =====
0: uvm_test_top.env.m_scoreboard [SCB] Total checks : 2000
0: uvm_test_top.env.m_scoreboard [SCB] Pass : 2000
0: uvm_test_top.env.m_scoreboard [SCB] Fail : 0
0: uvm_test_top.env.m_scoreboard [SCB] =====

```

해결 :

수신 register의 9번 비트인 busy 신호가 0으로 떨어질 때까지 반복해서 확인하는 do-while 추가
 -> 이렇게 되면 busy가 0일 때 루프를 빠져나오고, rdata에는 spi rdata가 담겨 있음.



느낀 점 :

Vivado(RTL)와 Vitis(C)를 활용한 AXI 기반의 시스템 통합 과정은 힘들었지만, 통합했을 때의 보드 동작이나 HW/SW를 넘나들며 시스템 전체를 설계하며 결과를 확인한 것은 매우 의미 있는 경험이라고 생각함.

감사합니다
